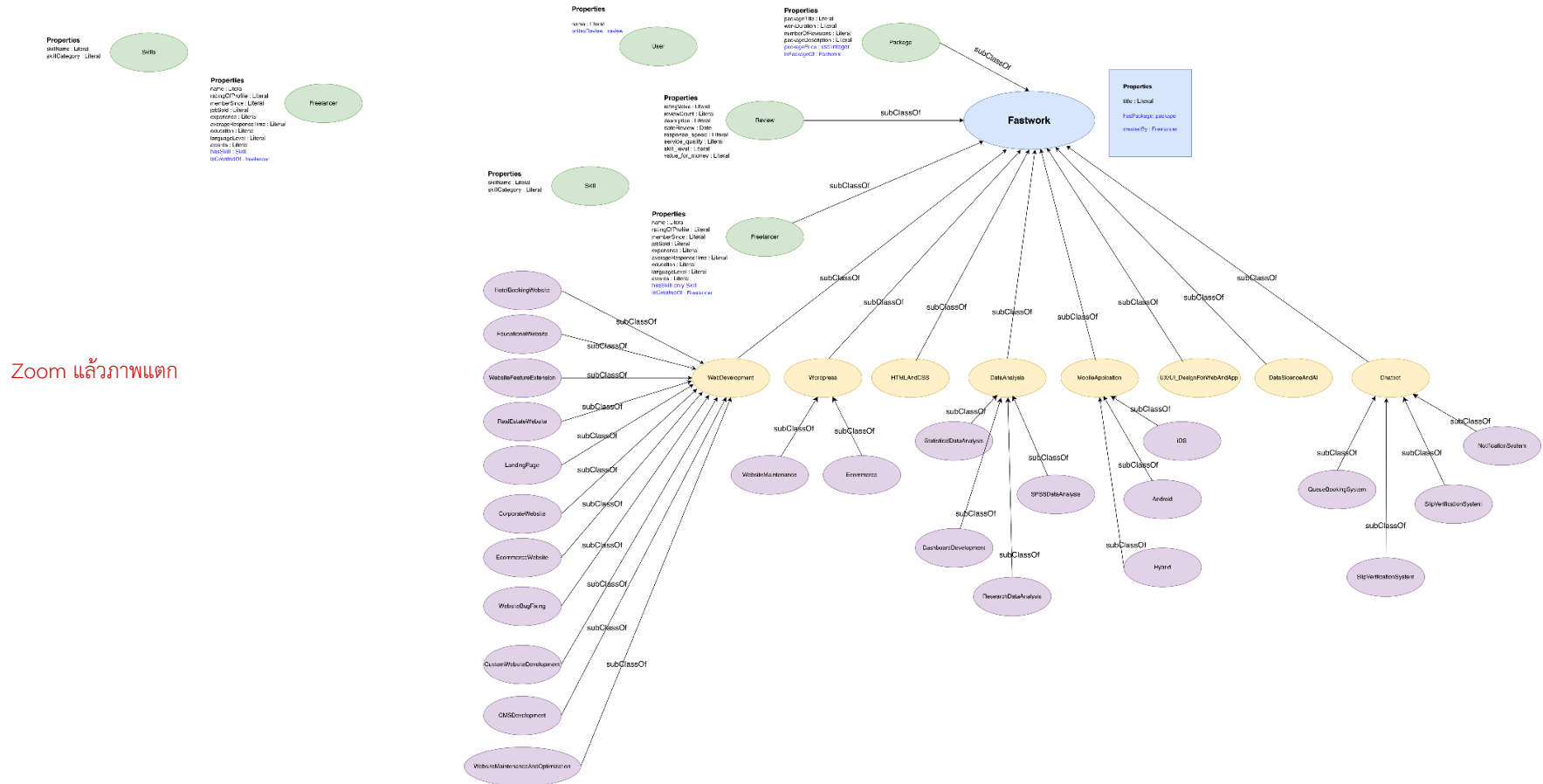
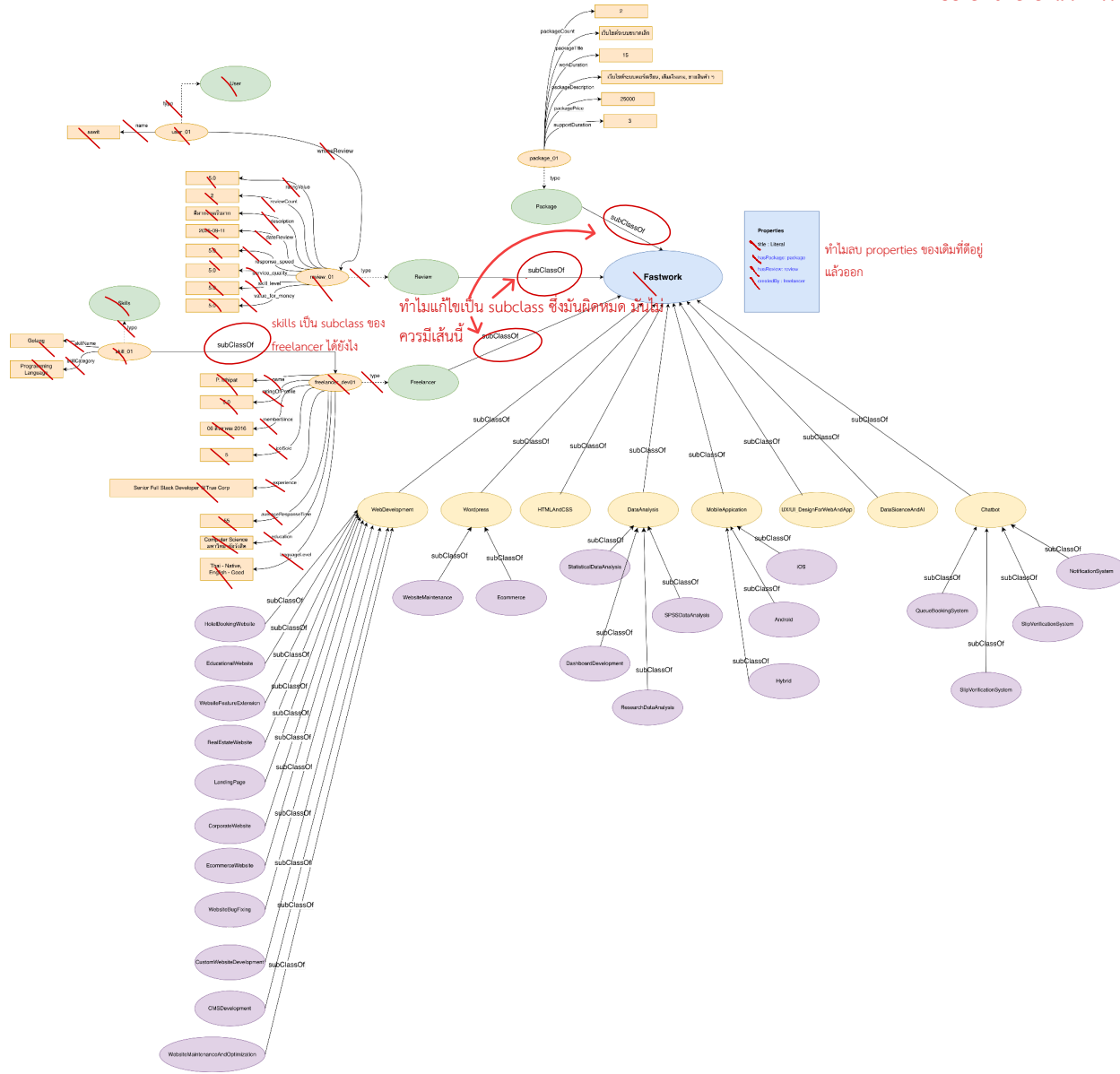


ASS04 - Ontology Design: Instance Level and OWL infer engine

Schema ที่แก้ไขเพิ่มเติม



Ontology ระดับ instance level (ชุมชนะคะ)



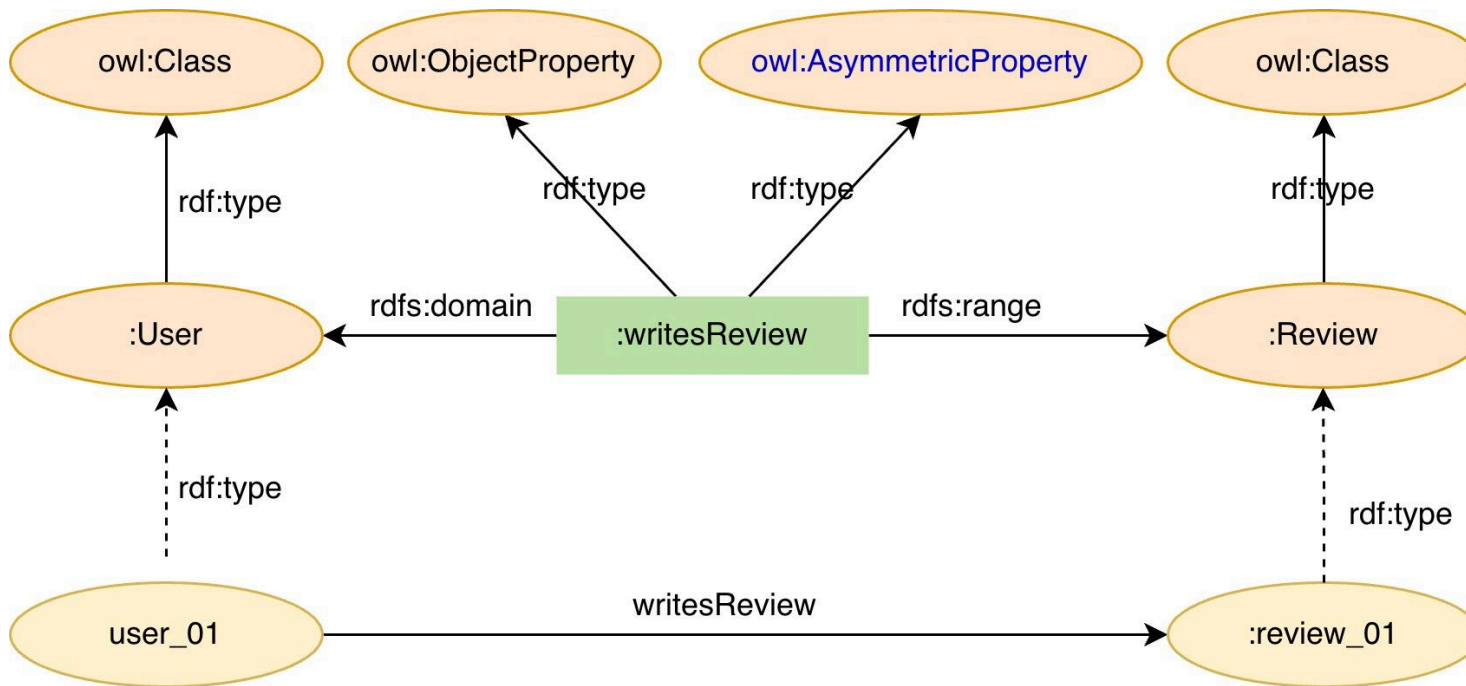
- ทำไมไม่มีตัวอย่าง Instance ของงานฟรีแลนซ์ และเชื่อมความสัมพันธ์กับคลาสอื่นเลย
- มีการใช้ object properties เป็น subclass อยู่หลายจุด เราควรทำความเข้าใจพื้นฐานของ subclass คืออะไรก่อน
- Schema level ไม่มีการระบุคุณสมบัติของ owl

หน้าออกแบบคุณสมบัติ OWL

เว้นวรรคหน่อย

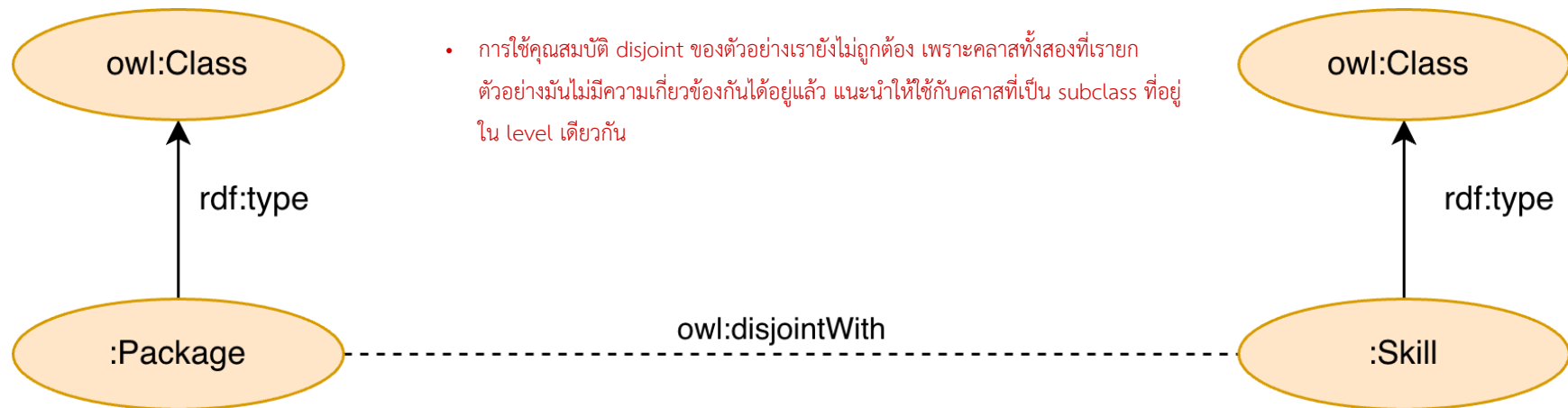
1. คุณสมบัติ AsymmetricProperty

- จากตัวอย่างควรใช้คุณสมบัติของ owl:inverseOf มากกว่า คุณสมบัติ Symmetric เพราะควรเป็นเส้นไปกลับ เช่น user01 writeReview review01 และ review01 createdBy user01 เช่นกัน



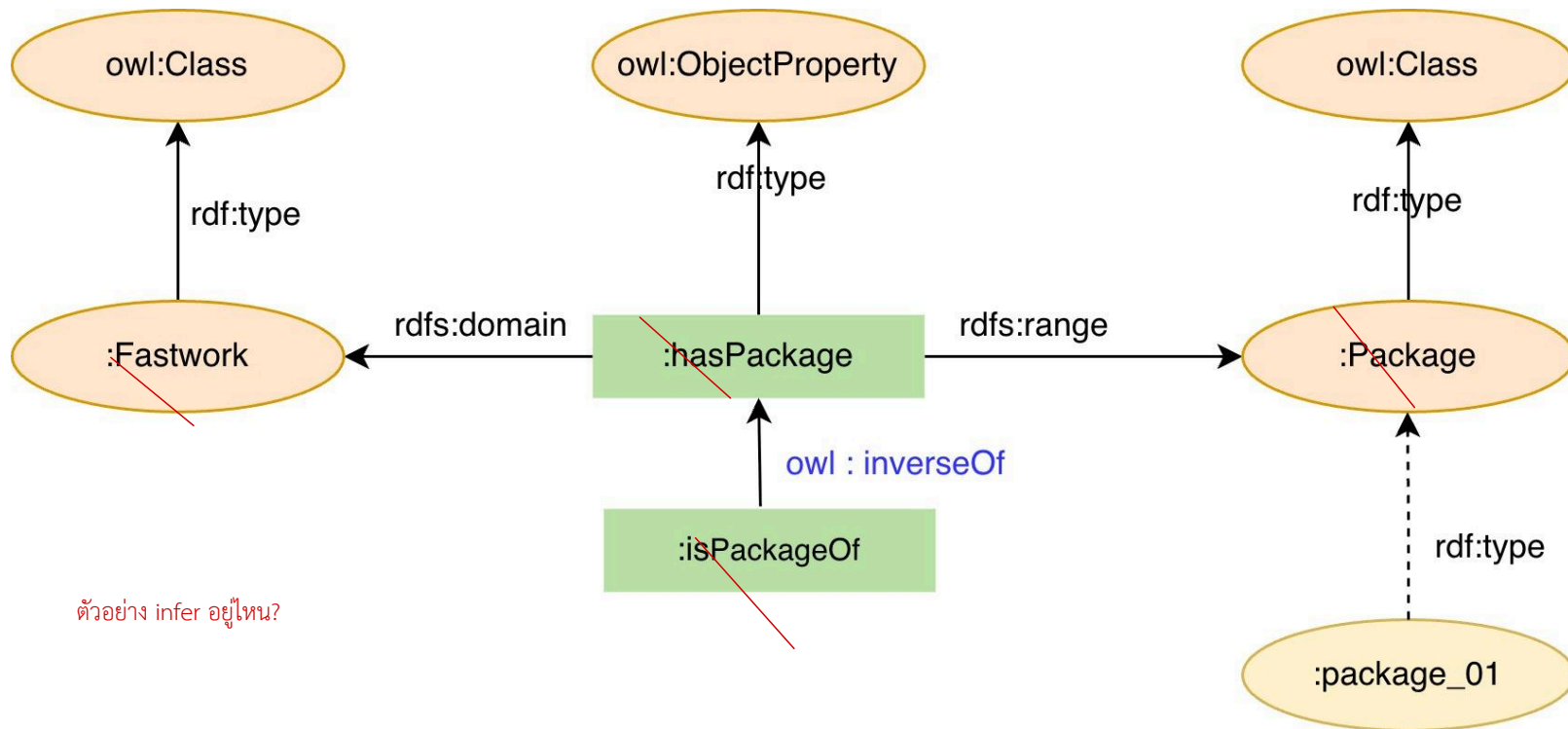
เรากำหนดให้คุณสมบัติ :writesReview (การเขียนรีวิว) เป็น owl:AsymmetricProperty เพื่อสร้างกฎว่า "**ความสัมพันธ์นี้เป็นแบบทิศทางเดียวและย้อนกลับไม่ได้ (คนเขียนรีวิวได้ แต่รีวิวจะกลับมาเขียนคนไม่ได้)**" โดยกำหนดให้เชื่อมโยงจากผู้ใช้งาน (:User) ไปยังข้อมูลปลายทางที่เป็นข้อความรีวิว (:Review) เช่น :user_01 ได้ถูกระบุว่าทำการเขียนรีวิวชื่อ :review_01 หากมีใครเพิ่มข้อมูลกลับฝั่งย้อนกลับ (เช่น ระบุให้ :review_01 เป็นคนเขียน :user_01) ระบบตรวจสอบตรรกะหลังบ้าน (Reasoner) จะฟ้อง Error ว่าข้อมูลขัดแย้งกัน (Inconsistent) ทันที ซึ่งจะช่วยให้โครงสร้างข้อมูลการรีวิวบนหน้าเว็บมีความสมเหตุสมผลตามความเป็นจริง ไม่เกิดปัญหาความสัมพันธ์ของข้อมูลสลับฝั่งจนทำให้ระบบประมวลผลผิดพลาด

2. คุณสมบัติ disjointWith



เรากำหนดให้คลาส `:Package` (แพ็คเกจบริการ) และคลาส `:Skill` (ทักษะความสามารถ) มีความสัมพันธ์ต่อกันแบบ `owl:disjointWith` (คลาสที่ไม่มีสมาชิกร่วมกัน) เพื่อสร้างกฎว่า **"ข้อมูลสองหมวดหมู่นี้แยกขาดจากกันอย่างเด็ดขาด สิ่งใดสิ่งหนึ่งจะเป็นทั้งแพ็คเกจ และทักษะพร้อมกันในเวลาเดียวกันไม่ได้"** เช่น หากเราสร้างข้อมูลชื่อ `:package_01` ขึ้นมาและระบุให้เป็นแพ็คเกจ (`:Package`) ไปแล้ว หากมีใครเผลอใส่ข้อมูลผิดพลาดโดยไปกำหนดทับให้ `:package_01` เป็นทักษะ (`:Skill`) ด้วย ระบบตรวจสอบตรรกะหลังบ้าน (Reasoner) จะฟ้อง Error ว่าข้อมูลขัดแย้งกัน (Inconsistent) ทันที ซึ่งจะช่วยป้องกันความสับสนในการจัดหมวดหมู่ข้อมูลบนเว็บ Fastwork ทำให้ระบบสามารถแยกแยะได้อย่างถูกต้องชัดเจนว่าอะไรคือแพ็คเกจบริการ และอะไรคือหมวดหมู่ทักษะของฟรีแลนซ์

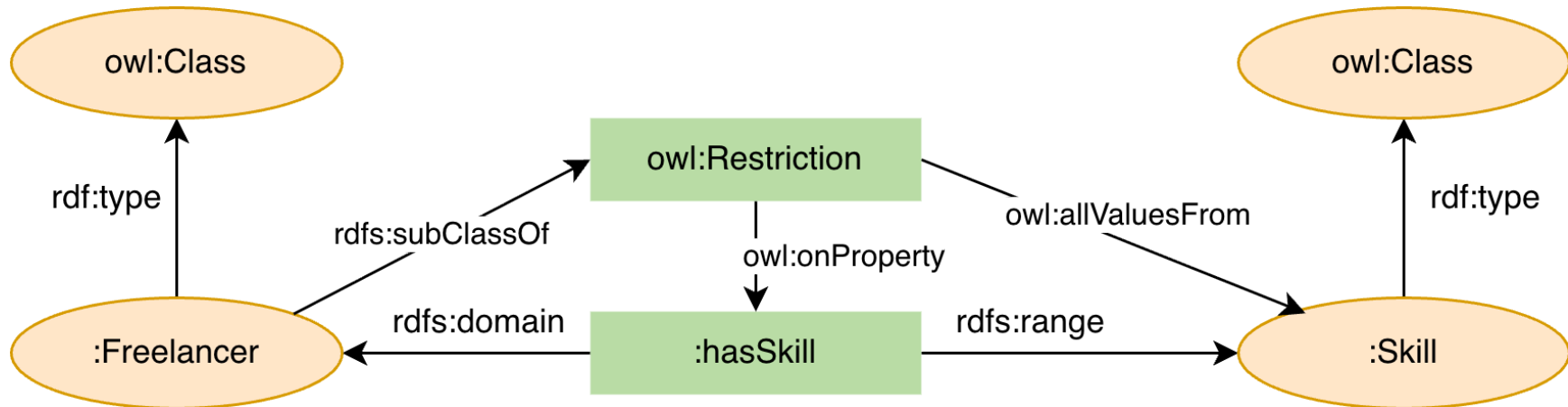
3. คุณสมบัติ inverseOf



เรากำหนดให้คุณสมบัติ `:hasPackage` (มีแพ็คเกจ) และ `:isPackageOf` (เป็นแพ็คเกจของ) มีความสัมพันธ์แบบ `owl:inverseOf` (คุณสมบัติผกผัน) เพื่อสร้างกฎว่า **"ความสัมพันธ์สองตัวนี้เป็นเส้นทางไป-กลับของกันและกันเสมอ"** โดยกำหนดให้เชื่อมโยงจากงานบริการ (`:Fastwork`) ไปยังข้อมูลปลายทางที่เป็นตัวแพ็คเกจ (`:Package`) เช่น หากเราป้อนข้อมูลลงไปเพียงฝั่งเดียวว่า งานรับทำเว็บ A มีแพ็คเกจ (`:hasPackage`) คือ `:package_01` ระบบตรรกะหลังบ้าน (Reasoner) จะทำการอนุมานและสร้างข้อมูลย้อนกลับให้เราโดยอัตโนมัติ (Infer) ว่า `:package_01` นั้นก็เป็นแพ็คเกจของ (`:isPackageOf`) งานรับทำเว็บ A ด้วยเช่นกัน ซึ่งจะช่วยลดภาระการป้อนข้อมูลซ้ำซ้อน ป้องกันปัญหาลิงก์ข้อมูลขาดหาย (Orphan data) และช่วยให้ระบบหน้าเว็บสามารถดึงข้อมูลเชื่อมโยงหากันได้อย่างสมบูรณ์แบบทั้งสองทิศทาง

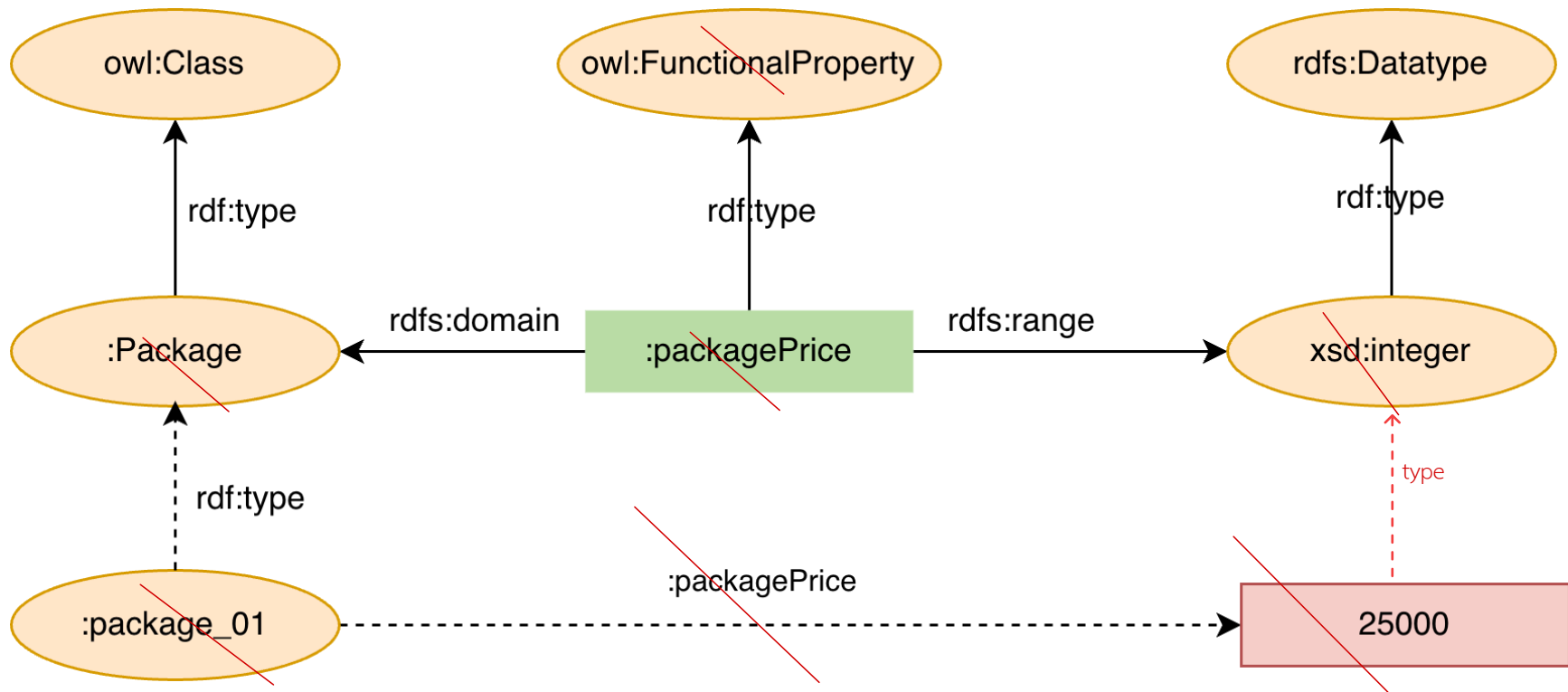
- จากคุณสมบัติของเราไม่จำเป็นต้องใช้คุณสมบัติ AllvalueFrom ก็ได้เพราะ range มีการระบุที่ชัดเจนอยู่แล้ว

4. คุณสมบัติ AllValuesFrom



เรากำหนดให้คลาส :Freelancer (ฟรีแลนซ์) มีเงื่อนไขการจำกัดคุณสมบัติแบบ owl:allValuesFrom (การบังคับกลุ่มข้อมูลปลายทาง) เพื่อสร้างกฎตรรกะว่า **"หากฟรีแลนซ์มีการระบุความสามารถผ่านคุณสมบัติ :hasSkill ข้อมูลปลายทางนั้นจะต้องดึงมาจากคลาส :Skill (ทักษะ) เท่านั้น จะไปดึงข้อมูลจากคลาสอื่นมาปะปนไม่ได้เด็ดขาด"** โดยสร้างเป็นเงื่อนไข (owl:Restriction) ควบคุมคุณสมบัติไว้ หากมีการป้อนข้อมูลที่ผิดเพี้ยนไปจากกรอบนี้ เช่น เผลอไประบุว่าฟรีแลนซ์คนหนึ่งมีทักษะความสามารถเป็น :package_01 (ซึ่งแท้จริงแล้วคือข้อมูลแพ็คเกจ ไม่ใช่หมวดหมู่ทักษะ) ระบบตรวจสอบตรรกะหลังบ้าน (Reasoner) จะฟ้อง Error ว่าข้อมูลขัดแย้งกัน (Inconsistent) ทันที กฎข้อนี้จึงช่วยดีกรอบการกรอกข้อมูลให้รัดกุม ป้องกันการใส่ข้อมูลข้ามหมวดหมู่ และรับประกันว่าในหน้าโปรไฟล์ของฟรีแลนซ์บนเว็บ Fastwork จะดึงมาแสดงได้เฉพาะรายการทักษะที่ถูกต้องตรงตามหมวดหมู่จริง ๆ เท่านั้น

5. คุณสมบัติ FunctionalProperty



เรากำหนดให้คุณสมบัติ **:packagePrice** (ราคาแพ็คเกจ) เป็น **owl:FunctionalProperty** เพื่อสร้างกฎว่า **"แพ็คเกจ 1 อัน จะต้องมียุติราคาที่แตกต่างกันเพียงราคาเดียวเท่านั้น"** โดยกำหนดให้เชื่อมโยงจากตัวแพ็คเกจไปยังข้อมูลปลายทางที่เป็นตัวเลข (xsd:integer) เช่น :package_01 ได้ถูกระบุราคาไว้ที่ 25000 บาท หากมีใครเพิ่มใส่ข้อมูลราคาอื่น (เช่น 50,000 บาท) ซ้อนทับเข้าไปในแพ็คเกจเดียวกันนี้ ระบบตรวจสอบตรรกะหลังบ้าน (Reasoner) จะฟ้อง Error ว่าข้อมูลขัดแย้งกัน (Inconsistent) ทันที ซึ่งจะช่วยให้หน้าเว็บแสดงราคาของแต่ละแพ็คเกจได้อย่างถูกต้อง ไม่เกิดปัญหาป้ายราคาซ้ำซ้อนให้ลูกค้าสับสน